



UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e
INTERACCIÓN HUMANO COMPUTADORA



REPORTE DE PRÁCTICA N° 01

NOMBRE COMPLETO: HERNÁNDEZ FIERROS SONIA

N° de Cuenta: 319194359

GRUPO DE LABORATORIO: 13

GRUPO DE TEORÍA: 07

SEMESTRE: 2027-1

FECHA DE ENTREGA LÍMITE: 2 de septiembre del 2026

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

Ventana cambia el color de fondo de forma random tomando rango de colores RGB y con una periodicidad de 2 segundos. (Verificar que al ejecutar el programa varias veces el orden de los colores si lo vean aleatorio y no siempre los mismos)

Tres letras iniciales de sus nombres creadas a partir de triángulos, todas las letras son del mismo color. Los dos ejercicios se muestran de forma simultánea y están en el mismo main

En este bloque se define el arreglo vertices[] dentro de la función CrearFiguras(). Estas coordenadas están normalizadas en el rango [-1.0, 1.0], que es el sistema que usa OpenGL

```
void CrearFiguras()
{
    GLfloat vertices[] = {
        //Letra S
        //Triangulo 1
        -0.3f, 0.5f, 0.0f,
        -0.3f, 0.4f, 0.0f,
        -0.5f, 0.4f, 0.0f,
        //triangulo 2
        -0.5f, 0.5f, 0.0f,
        -0.5f, 0.4f, 0.0f,
        -0.3f, 0.5f, 0.0f,
        //Triangulo 3
        -0.5f, 0.4f, 0.0f,
        -0.6f, 0.4f, 0.0f,
        -0.5f, 0.5f, 0.0f,
        //Triangulo 4
        -0.5f, 0.4f, 0.0f,
        -0.5f, 0.3f, 0.0f,
        -0.6f, 0.4f, 0.0f,
        //Triangulo 5
        -0.6f, 0.3f, 0.0f,
        -0.5f, 0.3f, 0.0f,
        -0.6f, 0.4f, 0.0f,
        //Triangulo 6
        -0.6f, 0.3f, 0.0f,
        96
        //Letra H
        //Triangulo 1
        -0.1f, 0.5f, 0.0f,
        98
        0.0f, 0.5f, 0.0f,
        99
        0.0f, 0.0f, 0.0f,
        100
        //triangulo 2
        101
        -0.1f, 0.0f, 0.0f,
        102
        -0.1f, 0.5f, 0.0f,
        103
        0.0f, 0.0f, 0.0f,
        104
        //Triangulo 3
        105
        0.0f, 0.3f, 0.0f,
        106
        0.0f, 0.2f, 0.0f,
        107
        0.1f, 0.2f, 0.0f,
        108
        //Triangulo 4
        109
        0.0f, 0.3f, 0.0f,
        110
        0.1f, 0.3f, 0.0f,
        111
        0.1f, 0.2f, 0.0f,
        112
        //Triangulo 5
        113
        0.1f, 0.5f, 0.0f,
        114
        0.1f, 0.0f, 0.0f,
        115
        0.2f, 0.0f, 0.0f,
        116
        //Triangulo 6
        117
        0.2f, 0.0f, 0.0f,
        118
        0.1f, 0.5f, 0.0f,
        119
        0.2f, 0.5f, 0.0f,
        120
        122
        //Letra F
        //Triangulo 1
        123
        0.4f, 0.5f, 0.0f,
        124
        0.4f, 0.0f, 0.0f,
        125
        0.5f, 0.0f, 0.0f,
        126
        //triangulo 2
        127
        0.4f, 0.5f, 0.0f,
        128
        0.5f, 0.0f, 0.0f,
        129
        0.5f, 0.5f, 0.0f,
        130
        //Triangulo 3
        131
        0.5f, 0.5f, 0.0f,
        132
        0.5f, 0.4f, 0.0f,
        133
        0.7f, 0.5f, 0.0f,
        134
        //Triangulo 4
        135
        0.7f, 0.5f, 0.0f,
        136
        0.7f, 0.4f, 0.0f,
        137
        0.5f, 0.4f, 0.0f,
        138
        //Triangulo 5
        139
        0.5f, 0.3f, 0.0f,
        140
        0.7f, 0.3f, 0.0f,
        141
        0.5f, 0.2f, 0.0f,
        142
        //Triangulo 6
        143
        0.5f, 0.2f, 0.0f,
        144
        0.7f, 0.2f, 0.0f,
        145
        0.7f, 0.3f, 0.0f,
        146
        147
    };
```

Se declaran las variables r, g y b de tipo GLfloat, inicializadas todas en 0.0f, es decir color negro. También se llama a a srand((unsigned int)time(NULL)), se hace antes de entrar al while, ya que time(NULL) no es un valor que se quede fijo, cada segundo cambia. (unsigned int)time(NULL) constituye la semilla, por lo que la semilla cambia por lo tanto cada segundo, esto es lo que garantiza la aleatoriedad, que la semilla cambie, ya que más adelante al usar rand() al menos dos veces, este no se va a replicar nunca.

```

276     GLfloat r = 0.0f; //es buena práct
277     GLfloat g = 0.0f;
278     GLfloat b = 0.0f;
279     //se empieza con el color negro
280
281     srand((unsigned int)time(NULL));
282
283
284
285
286
287     double tiempo1 = 0.0; //empezamos

```

Dentro del while se calcula que ya haya transcurrido más de dos segundos, si esto sucede, entra al if, donde se generan los tres números aleatorios entre el rango 0.0f y 1.0f. Para crear los flotantes r, g y b, por ejemplo si rand() nos devuelve 12897, al dividirse por ejemplo entre 1000 nos da de residuo 897, que a su vez al dividirse entre 999.0f da 0.89f lo cual ya es aceptable, ya que debe valer entre 1.0f y 0.0f.

No es forzoso hacer el módulo de 1000 y dividir entre 999, en realidad se puede hacer el modulo de cualquier número y dividir entre ese mismo número menos una unidad. El chiste es obtener un número random entre 1.0f y 0.0f

```

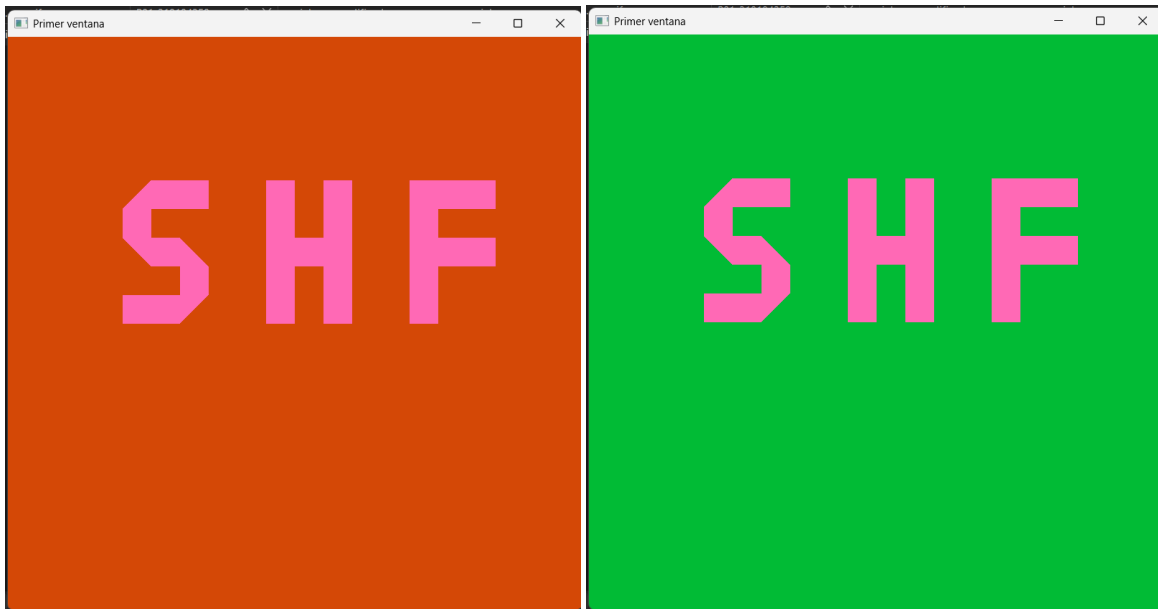
while (!glfwWindowShouldClose(mainWindow))
{
    //Recibir eventos del usuario
    glfwPollEvents();

    double tiempoActual = glfwGetTime(); //en c
    if (tiempoActual - tiempo1 >= 2.0) //han tr
    {
        r = (GLfloat)(rand() % 1000) / 999.0f;
        g = (GLfloat)(rand() % 256) / 255.0f;
        b = (GLfloat)(rand() % 589) / 588.0f;
        // las 4 formas fu
        tiempo1 = tiempoActual;
    }
    //Limpiar la ventana
    glClearColor(r, g, b, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT);
}

```

Haciendo el conteo de la cantidad de vértices que se tiene entre los 26 triángulos, se tienen 78 vértices. No es perceptible a simple ojo, pero realmente se están viendo muchos triángulos de color rosa, no letras.





Esta parte define el color rosa.

```
24 static const char* fShader = " \n\  
25 #version 330 \n\  
26 out vec4 color; \n\  
27 void main() \n\  
28 { \n\  
29     color = vec4(1.0f, 0.41f, 0.71f, 1.0f); \n\  
30 }; //color rosa
```

2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla

hacerlo más de una vez es innecesario. Así que para conseguir obtener los valores de los enteros R, G, B se usó un poco de aritmética para conseguir valores menores a 1.0f y mayores a 0.0f

- b. Comentarios generales: Faltó explicar a detalle, ir más lento en alguna explicación, otros comentarios y sugerencias para mejorar desarrollo de la práctica

Como sugerencia, sería útil que las explicaciones de nuevas funciones o conceptos se dieran a un nivel más general primero (qué hace la función y por qué se usa), y no necesariamente línea por línea desde el inicio, esto ayudaría a entender el panorama completo antes de entrar al detalle técnico, sobre todo cuando se trata de funciones que no se habían visto antes en el curso.

- c. Conclusión.

Se reforzaron los conocimientos del manejo básico de OpenGL mediante GLFW y GLEW, mediante un ejercicio sencillo como es la creación de letras mediante triángulos y el uso de shaders (vertex y fragment). Y se reforzó el cómo se debe de cambiar un fondo, recordando como crear aleatoriedad en lenguaje C.

1. Bibliografía en formato APA

cppreference.com. (s.f.). `std::rand`, `std::srand`. Recuperado de <https://en.cppreference.com/c/numeric/random/rand>

cppreference.com. (s.f.). `std::srand`. Recuperado de <https://en.cppreference.com/c/numeric/random/srand>

GeeksforGeeks. (2025, 23 de julio). `rand()` and `srand()` in C/C++. Recuperado de <https://www.geeksforgeeks.org/cpp/rand-and-srand-in-ccpp/>

GeeksforGeeks. (2025, 23 de julio). `rand()` function in C. Recuperado de <https://www.geeksforgeeks.org/c/c-rand-function/>

GeeksforGeeks. (2026, 6 de agosto). `time()` function in C. Recuperado de <https://www.geeksforgeeks.org/c/time-function-in-c/>

Khronos Group. (2006). `glClearColor` [Documentación de referencia de OpenGL]. Recuperado de <https://registry.khronos.org/OpenGL-Refpages/gl4/html/glClearColor.xhtml>